

The `gfs_dynamical_core.jax` Technical Manual

A differentiable spectral primitive-equation dynamical core in JAX

`gfs_dynamical_core / docs/gfs_jax_manual/`

July 16, 2026

Abstract

`gfs_dynamical_core.jax` is a dry, spectral, primitive-equation atmospheric dynamical core — a re-implementation of the hydrostatic GFS/SHTNS Fortran core — written entirely as pure JAX functions. Because every operation is a traceable JAX primitive, the whole model is `jit`-compilable, `vmap`-vectorisable over ensembles, and, above all, *differentiable*: one can take exact derivatives of any forecast quantity with respect to the initial state, the parameters, or the forcing, in either reverse (adjoint) or forward (tangent-linear) mode, without writing a separate adjoint model. This manual documents the architecture and numerics of the core, shows how to build and run a Held–Suarez model on top of it, and — the main event — explains what differentiability buys you and how to use it, with six short, runnable example scripts (sensitivities, Jacobians, ensembles, variational optimisation, singular vectors). It is written for a graduate student or research scientist who knows atmospheric dynamics and some Python, and wants to *use* the core, not just read about it.

Contents

I	The dynamical core	3
1	Introduction and scope	3
2	Architecture and data flow	3
3	Environment and conventions	4
4	State representation	4
5	Spectral transforms	5
6	The primitive-equation dynamics	6
7	Time integration	6
II	Building and running a model	7
8	Building a Held–Suarez model	7
9	The Held–Suarez forcing	7
10	Integrating in time with <code>lax.scan</code>	7

III	Differentiable modelling	8
11	What differentiability buys a GCM	8
11.1	What the gradient is, and with respect to what	9
11.2	Reverse mode vs. forward mode	9
12	Six capabilities, six scripts	9
12.1	Example 1 — forward run (<code>jit + lax.scan</code>)	9
12.2	Example 2 — ensembles with <code>vmap</code>	9
12.3	Example 3 — adjoint sensitivity with <code>grad</code>	9
12.4	Example 4 — Jacobians (<code>jacfwd</code> / <code>jacrev</code>)	10
12.5	Example 5 — variational optimisation (a 4D-Var flavour)	10
12.6	Example 6 — optimal perturbations / singular vectors (<code>jvp+vjp</code>)	11
13	Numerical caveats when differentiating	11
14	Extending the core: non-differentiable kernels	12
	API quick reference	12
	Further reading	12

Part I

The dynamical core

1 Introduction and scope

The core solves the **dry hydrostatic primitive equations** on the sphere in **spherical-harmonic (spectral) space**, on a hybrid sigma–pressure vertical coordinate, with a semi-implicit IMEX Runge–Kutta time scheme. It is a faithful port of an operational-lineage Fortran core (GFS dynamics, SHTNS transforms): the numerics — the vertical discretisation, the semi-implicit gravity-wave treatment, the hyper-diffusion, the dry-mass fixer — are reproduced, and validated against the Fortran reference. What is new is the *substrate*: the code is pure JAX, so it inherits automatic differentiation, JIT compilation, and vectorisation.

Why re-write a working Fortran core in JAX? Not for raw speed (on CPU it is comparable). The payoff is *composability with autodiff and hardware acceleration*: gradients of a forecast metric with respect to millions of inputs at the cost of a single extra integration (§11); ensembles as a batch axis (§12.2); and the ability to embed the core inside a larger differentiable program (a machine-learned parametrisation, a variational assimilation, an optimisation loop) and train the whole thing end-to-end. None of these are available from a conventional Fortran core without writing and maintaining separate tangent-linear and adjoint models by hand.

The core is deliberately *small* and readable: four modules, ~1800 lines (Table 1).

Table 1: The four modules of `gfs_dynamical_core.jax`.

module	responsibility
<code>states.py</code>	the state containers (<code>SpectralState</code> , <code>GridState</code> , <code>GridGradients</code> , <code>SpectralTendencies</code>) as <code>flax</code> pytrees.
<code>transforms.py</code>	spherical-harmonic transforms (scalar and spin-weighted), precompute-kernel caching, grid geometry, triangular-truncation & reality enforcement.
<code>dynamics.py</code>	the primitive-equation right-hand side: pressure diagnostics, vertical velocities, pressure-gradient force, vertical advection, energy conversion, tendency assembly, dry-mass fixer.
<code>stepper.py</code>	time integration: the 3-stage IMEX-RK scheme, the semi-implicit gravity-wave solve, hyper-diffusion/Rayleigh damping, and the physics-injection entry point <code>advance_with_tendencies</code> .

2 Architecture and data flow

One model time step is a loop through spectral and grid space:

1. Start from the **spectral state** S (vorticity, divergence, temperature, $\ln p_s$, tracers — as spherical-harmonic coefficients).
2. **Inverse transform** $S \rightarrow$ grid state G (winds u, v , temperature, $\ln p_s$, tracers) *and* the horizontal gradients needed by the dynamics (`spectral_to_grid`).

3. Evaluate the nonlinear **primitive-equation tendencies** in grid space (`full_dynamics_step`): products like $u\zeta$, the pressure-gradient force, vertical advection — everything that is cheap and local on the grid.
4. **Forward transform** the grid tendencies back to spectral tendencies (`grid_to_spectral_tendencies`), where the linear terms (Laplacians, gravity waves) are diagonal and cheap.
5. **Advance** in spectral space with the semi-implicit IMEX-RK scheme, apply diffusion, optionally inject physics (`advance`).

The grid $\leftrightarrow$ spectral round trip per stage is the defining feature of a spectral transform model: nonlinear products are done where they are local (grid), linear operators where they are diagonal (spectral). The core does this three times per step (a 3-stage RK).

3 Environment and conventions

Float64 is mandatory; pin the CPU backend on Apple Silicon. The spectral dynamics are numerically stiff and single precision corrupts them silently. Set, *before importing jax*:

```
import os
os.environ["JAX_ENABLE_X64"] = "True"
os.environ.setdefault("JAX_PLATFORMS", "cpu") # Metal (Apple GPU) has no float64
```

A cluster exporting `JAX_PLATFORMS=cuda` still wins (note the `setdefault`). Getting this wrong gives either a crash or, worse, quietly degraded numbers.

Vertical indexing: bottom-to-top. Level index $k = 0$ is the surface, $k = n_{\text{lev}}$ the model top. All fields, coordinate coefficients `ak/bk`, and diagnostics use this convention. (The Fortran reference is top-to-bottom in places; the port standardises on bottom-to-top and the semi-implicit setup flips internally.)

Grid geometry. Gauss–Legendre (GL) sampling: $n_{\text{lat}} = L$, $n_{\text{lon}} = 2L - 1$, where L is the spherical-harmonic band-limit. Latitudes are Gaussian quadrature points, north-to-south. The *physical* triangular truncation T (de-aliasing) defaults to the GFS convention $T = \lfloor n_{\text{lon}}/3 - 2 \rfloor$; spectral arrays are stored on the full $(L, 2L - 1)$ rectangle but coefficients with $\ell > T$ or $|m| > T$ are zeroed after every forward transform.

Setup-time constants come from climt. The vertical coordinate (`ak,bk`) and Earth constants (R_d , c_p , Ω , g , radius, model-top pressure) are fetched once from `climt.get_grid(...)` at build time (§8). Nothing in the time loop depends on `climt`.

4 State representation

States are `flax.struct` dataclasses — i.e. JAX *pytrees*: their array fields are traced leaves (so `jit/grad/vmap` see through them), while scalar configuration fields can be marked static. The two central containers (`states.py`):

```
@struct.dataclass
class SpectralState: # spherical-harmonic coefficients (complex128)
    vorticity: # (n_lev, L, 2L-1)
    divergence: # (n_lev, L, 2L-1)
    temperature: # (n_lev, L, 2L-1)
```

```

log_surface_pressure: # (L, 2L-1)
tracers: # (n_tracers, n_lev, L, 2L-1)

@struct.dataclass
class GridState: # physical fields on the Gaussian grid (float64)
    u, v, temperature, vorticity, divergence: # (n_lev, n_lat, n_lon)
    log_surface_pressure: # (n_lat, n_lon)
    tracers: # (n_tracers, n_lev, n_lat, n_lon)

```

The prognostic variables are the *vorticity-divergence* form of the momentum equations (not u, v): this makes the linear terms diagonal in spectral space. `GridGradients` carries the horizontal derivatives of $\ln p_s$, temperature and tracers (produced for free by the spin-weighted transform, §5); `SpectralTendencies` mirrors `SpectralState` with $\partial/\partial t$ of each field. Because states are pytrees, an “ensemble of states” is just the same pytree with a leading batch axis, and a “perturbation to a state” is a pytree of the same shape — both facts we lean on heavily in Part III.

5 Spectral transforms

The transforms (`transforms.py`) are built on `s2fft`. Three points matter for a user.

Scalar and spin-weighted transforms. Temperature, $\ln p_s$ and tracers are ordinary scalar fields (spin-0). The winds are handled with **spin-1** transforms: from vorticity ζ and divergence D ,

$$F_{\ell m}^{(1)} = \frac{R}{\sqrt{\ell(\ell+1)}} (D_{\ell m} + i\zeta_{\ell m}), \quad u = \text{Im } f^{(1)}, \quad v = -\text{Re } f^{(1)}, \quad (1)$$

and inversely, `grid_to_spectral` uses *both* spin +1 and spin −1 forward transforms to recover $D_{\ell m}$ and $\zeta_{\ell m}$ exactly for all m (a single transform loses information for $m \geq 1$, where D, ζ are complex). The same spin-1 machinery yields the horizontal *gradients* of scalars (temperature, $\ln p_s$) as a by-product, which the pressure-gradient and advection terms need.

Precompute kernels. `s2fft`’s on-the-fly Wigner recursions dominate cost (~ 7 ms/transform at $L = 64$). The core instead uses *precompute kernels* — each transform becomes a dense contraction (~ 0.6 ms, $\sim 12\times$ faster, matching the recursions to $\sim 10^{-12}$). Kernels are cached per $(L, \text{sampling}, \text{spin}, \text{direction})$ and, under `jit`, are baked in as constants. Kernel construction is *not jit*-traceable, so it must happen eagerly *before* tracing:

```

from gfs_dynamical_core.jax.transforms import prebuild_kernels
prebuild_kernels(L, "gl") # call once at setup, before any jit-compiled step

```

Triangular truncation and the reality condition. After every forward transform, `enforce_triangular_t` (i) zeroes coefficients with $\ell > T$ or $|m| > T$, and (ii) enforces the reality condition $f_{\ell, -m} = (-1)^m \overline{f_{\ell, +m}}$ with $\text{Im } f_{\ell, 0} = 0$.

Why the reality condition is not optional. `s2fft` stores the full $(\ell, \pm m)$ rectangle and treats fields as *general complex* functions. Round-off (mainly in the dual-spin forward combinations) seeds a tiny violation of $f_{\ell, -m} = (-1)^m \overline{f_{\ell, +m}}$ — which is an imaginary-valued grid field. That imaginary field evolves as an *undamped tangent-linear perturbation* of the flow: it e-folds with the jet’s fastest instability (~ 5.5 h) and, coupling back through the nonlinear terms, blows the model up around day 7–8. SHTNS/Fortran store only $m \geq 0$ and so are symmetric by construction; enforcing the symmetry at every forward transform makes

the JAX core do the same, and all subsequent real-coefficient linear operations preserve it. This was one of the subtle bugs found and fixed during the port.

6 The primitive-equation dynamics

`dynamics.py` evaluates the right-hand side. On the hybrid coordinate the interface pressures are $p_k = a_k + b_k(p_s - p_{\text{top}})$ (`compute_pressure_diagnostics`), from which layer thicknesses Δp , layer-mean pressures, and the geometric factors α , $\delta \ln p$ follow. The pipeline (`full_dynamics_step`) is:

1. **Vertical velocities** (`compute_vertical_velocities`): vertically integrate the continuity equation to get the surface-pressure tendency $\partial_t \ln p_s$, the coordinate vertical velocity $\dot{\eta}$ on interfaces, and $\omega = d \ln p / dt$ on layers.
2. **Pressure-gradient force** (`compute_pressure_gradient_force`): the horizontal PGF from virtual temperature, $\ln p_s$ gradients and the surface-geopotential gradients, integrated through the hybrid layers.
3. **Vertical advection**: second-order centred for u, v, T (`compute_vertical_advection`); a positive-definite [Thuburn \(1996\)](#) flux-limited scheme for tracers (`compute_vertical_advection_tracers`) so that mixing ratios cannot go negative.
4. **Energy conversion** $\kappa \omega T_v / [1 + (c_{\text{vap}}/c_p - 1)q]$.
5. **Assemble tendencies** (`assemble_grid_tendencies`): the flux form u -flux $= u(\zeta + f) + (\text{vadv}_v - \text{PGF}_y)$ etc. with Coriolis $f = 2\Omega \sin \phi$, temperature and tracer tendencies, and kinetic energy.

`grid_to_spectral_tendencies` then maps these back: $\partial_t \zeta = -\nabla \cdot \text{flux}$, $\partial_t D = \nabla \times \text{flux} - \nabla^2(\text{KE})$. Optionally a **dry-mass fixer** (`compute_dry_mass_fixer`) rescales the $\ln p_s$ tendency each step so the global dry-air mass is conserved.

7 Time integration

The stepper (`stepper.py`) is a **3-stage IMEX Runge–Kutta** scheme: the fast linear gravity-wave terms are treated *implicitly* (so the time step is not limited by the sound/gravity-wave speed), the nonlinear advective terms *explicitly*.

Semi-implicit gravity-wave solve. The gravity-wave terms are linearised about a reference state ($T_{\text{ref}} = 300 \text{ K}$, $p_{s,\text{ref}} = 800 \text{ hPa}$), giving four wavenumber-independent operators built once by `init_semi_implicit_matrices`: `amhyb` (geopotential PGF acting on T_v), `bmhyb` (energy conversion acting on D), `tor_hyb` ($\ln p_s$ in the PGF) and `svhyb` ($\ln p_s$ tendency from D). With $Y_m = \text{tor_hyb} \otimes \text{svhyb} + \text{amhyb} \cdot \text{bmhyb}$ the implicit solve at each stage and each degree ℓ inverts

$$(I + (c \Delta t)^2 \ell(\ell + 1) Y_m)^{-1}, \quad (2)$$

pre-computed as `d_hyb_m` of shape $(3, L, n_{\text{lev}}, n_{\text{lev}})$ (three implicit coefficients c , one $n_{\text{lev}} \times n_{\text{lev}}$ inverse per degree). Vorticity and tracers are advanced explicitly; divergence, temperature and $\ln p_s$ go through the implicit solve.

Diffusion. `init_diffusion_operators` builds a spectral hyper-diffusion operator $\propto [\ell(\ell + 1)/\ell_{\text{max}}(\ell_{\text{max}} + 1)]^4$ (∇^8 , `ndiss=8`), a height-dependent enhancement profile, and an optional upper-level Rayleigh damping. These are applied as an implicit linear factor at the end of each step.

The two entry points. `advance(...)` performs one dynamics-only step. `advance_with_tendencies(...)` performs the same dynamics step and then applies a *time-split physics increment*:

```
advance_with_tendencies(spec_state, phys_tends, phis_grads, dyn_config,
                        trans_config, stepper_config, latitudes,
                        gauss_weights=None, pdryini=None, spec_tends=None)
```

`phys_tends` is a grid-space `PhysicsTendencies` (e.g. Held–Suarez forcing, or a machine-learned parametrisation) transformed to spectral internally; `spec_tends` is a spectral-space increment added *directly* (the injection point for stochastic schemes such as SPPT/SKEB). With both `None` it is exactly `advance`. The increment is applied as $S_{\text{new}} = S_{\text{dyn}} + \Delta t$ (increment).

Part II

Building and running a model

8 Building a Held–Suarez model

The core provides the dynamics; a *model* adds a vertical coordinate, constants, and a forcing. The reference builder is `docs/gfs_jax_manual/examples/_model.py`. It (i) fetches `ak,bk` and Earth constants from `climt`; (ii) assembles the `DynamicsConfig`, `TransformConfig`; (iii) pre-builds transform kernels; (iv) builds the semi-implicit and diffusion operators; and (v) packs everything into a single pytree `ModelBundle`:

```
bundle = build_model("T21") # L=32, T=21, dt=1800 s, 20 levels
# ModelBundle carries: dyn_config, trans_config, stepper_config, latitudes,
# gauss_weights, phis_grads, dt, n_lev. Array leaves are differentiable;
# dt / n_lev are static pytree aux, so one jit graph is reused across steps.
```

A single deterministic step is then just the grid→tendency→advance sequence:

```
def step(bundle, spec_state):
    grid, _ = spectral_to_grid(spec_state, bundle.trans_config)
    phys = hs_tendencies(grid, bundle) # Held-Suarez forcing
    return advance_with_tendencies(
        spec_state, phys, bundle.phis_grads, bundle.dyn_config,
        bundle.trans_config, bundle.stepper_config, bundle.latitudes,
        bundle.gauss_weights, None)
```

9 The Held–Suarez forcing

[Held & Suarez \(1994\)](#) is the standard idealised forcing for testing dynamical cores: Newtonian relaxation of temperature towards a prescribed radiative equilibrium $T_{\text{eq}}(\phi, p)$ at a latitude/height-dependent rate, plus Rayleigh friction of low-level winds. It has no moisture and no radiation code, so it is cheap, reproducible, and isolates the dynamics. In the core’s terms it is just a function `hs_tendencies(grid_state, bundle)` returning a `PhysicsTendencies`; swapping in a different (even machine-learned) physics is a matter of returning the same container.

10 Integrating in time with `lax.scan`

Do *not* write a Python `for` loop over steps: it would unroll into a giant graph. The idiomatic long integration is a `jax.lax.scan`, compiled once with `jax.jit`:

```

@partial(jax.jit, static_argnums=(2,))
def run(bundle, spec0, n_steps):
    def body(spec, _):
        spec = step(bundle, spec)
        # ... optionally compute and stack a diagnostic here ...
        return spec, diagnostic
    spec_final, trace = jax.lax.scan(body, spec0, None, length=n_steps)
    return spec_final, trace

```

This is **Example 1** (`ex01_run_forward.py`): a 6-day Held–Suarez spin-up from an isothermal rest state, watching kinetic energy grow as the jet and baroclinic eddies develop (Figure 1). For very long or memory-heavy integrations, wrap `body` in `jax.checkpoint` to trade compute for memory (§13).

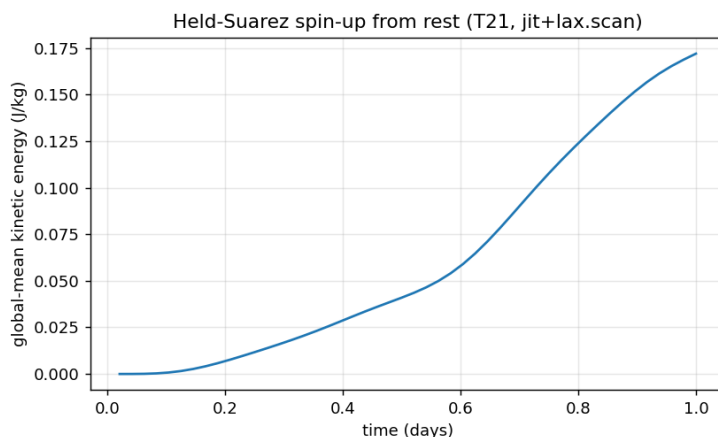


Figure 1: Example 1: global-mean kinetic energy during a Held–Suarez spin-up from rest (T21), computed by a single jit-compiled `lax.scan`.

Part III

Differentiable modelling

11 What differentiability buys a GCM

Every quantity the core computes is a composition of differentiable JAX primitives. Consequently any scalar *metric* J of a forecast — a global mean, a regional average, a misfit to observations, a forecast error — is a differentiable function of whatever it depends on: the initial state x_0 , the model parameters θ , the forcing. JAX gives the derivatives exactly (to machine precision, not by finite differences) and cheaply.

The adjoint, for free. In a conventional GCM, obtaining $\partial J / \partial x_0$ (the sensitivity of a forecast metric to the initial state) requires a hand-written *adjoint model*: a separate code, derived line-by-line from the forward model, that must be re-derived and re-validated every time the forward model changes. It is one of the most labour-intensive artefacts in numerical weather prediction. Here it is one function call, `jax.grad(J)`, generated automatically from the forward code and therefore *always* consistent with it. The tangent-linear model is likewise `jax.jvp`; the adjoint is `jax.vjp`.

11.1 What the gradient is, and with respect to what

Let $J : \mathcal{X} \rightarrow \mathbb{R}$ map an input (an initial-state field, a vector of parameters, ...) to a scalar. `jax.grad(J)(x)` returns $\nabla_x J$, an object of the *same pytree shape as x* — e.g. if x is an initial temperature field of shape $(n_{\text{lev}}, n_{\text{lat}}, n_{\text{lon}})$, then $\nabla_x J$ is a field of that shape whose value at each point says how much J changes per unit change of the initial temperature there. The gradient is taken with respect to *whatever you pass as the differentiated argument*, and nothing else is moved.

11.2 Reverse mode vs. forward mode

For a map $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ autodiff comes in two flavours:

- **Reverse mode** (`grad`, `jacrev`, `vjp`) propagates *cotangents* backwards. Cost $\propto m$ (number of outputs). Use it when outputs are few and inputs many — the usual case $m = 1$ (a scalar metric, so one backward pass gives the full gradient over all n inputs). This is the adjoint.
- **Forward mode** (`jacfwd`, `jvp`) propagates *tangents* forwards. Cost $\propto n$ (number of inputs). Use it when inputs are few and outputs many. This is the tangent-linear model.

A full Jacobian is $\min(m, n)$ such passes; `jax.jacrev` and `jax.jacfwd` build it for you and agree to rounding (Example 4).

12 Six capabilities, six scripts

Each script lives in `docs/gfs_jax_manual/examples/` and runs at T21 in a few minutes on a laptop CPU. Run e.g. `python -m docs.gfs_jax_manual.examples.ex03_gradient_sensitivity`.

12.1 Example 1 — forward run (`jit + lax.scan`)

Covered in §10: compile the whole time loop once and integrate. The foundation everything else builds on.

12.2 Example 2 — ensembles with `vmap`

A Fortran core runs one state per process; an ensemble means many executables. Here the model is a pure function of its state, so `jax.vmap` vectorises the *same compiled* integration over a batch of perturbed initial conditions — the ensemble is one array axis:

```
ens0 = jax.vmap(perturb)(keys) # batched SpectralState
ke = jax.vmap(lambda s: rollout(bundle, s, n_steps))(ens0) # one call, all members
```

`ex02_ensemble_vmap.py` launches an 8-member ensemble from tiny initial-temperature perturbations and shows the members diverging (chaotic growth) after a couple of days. The same axis could be a batch of *parameter* settings or *forcings*.

12.3 Example 3 — adjoint sensitivity with `grad`

The capability a Fortran core cannot give without a hand-coded adjoint. Define a scalar forecast metric $J = \text{global-mean kinetic energy after 12 h}$, as a function of the *initial temperature field* T_0 , and compute $\partial J / \partial T_0$ with one call:

```
J, dJ_dT0 = jax.value_and_grad(metric)(T0) # the adjoint sensitivity map
```

$\partial J / \partial T_0[k, j, i]$ is the change in later kinetic energy per unit initial-temperature perturbation at each point (Figure 2). This is a one-integration-cost answer to “where does the initial state most control this forecast metric?” — the question at the heart of targeted observations, sensitivity analysis, and data assimilation.

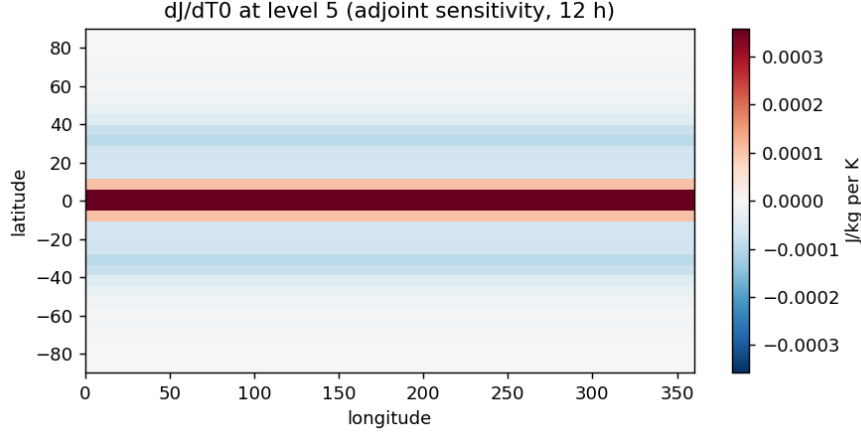


Figure 2: Example 3: adjoint sensitivity $\partial J / \partial T_0$ of a 12-hour kinetic-energy metric to the initial temperature field, at the most sensitive model level. Structure is near-zonal because the (lightly spun-up) base state is near-zonally-symmetric; the equatorial band dominates. Produced by `jax.grad`, at the cost of $\sim$ one forward integration.

12.4 Example 4 — Jacobians (`jacfwd` / `jacrev`)

For a small vector-to-vector map $f : \mathbb{R}^k \rightarrow \mathbb{R}^m$ (k initial-perturbation amplitudes $\rightarrow m$ forecast diagnostics), both modes build the same Jacobian:

```
Jr = jax.jacrev(f)(theta0) # reverse: cheap when m < k
Jf = jax.jacfwd(f)(theta0) # forward: cheap when k < m
# max|Jr - Jf| ~ 1e-17 (they agree)
```

$\partial f_j / \partial \theta_i$ is the tangent-linear response of diagnostic j to perturbation i — a tangent-linear model restricted to a chosen input/output subspace. `ex04_jacobian.py` prints the 3×4 matrix and verifies forward/reverse agreement.

12.5 Example 5 — variational optimisation (a 4D-Var flavour)

Because the forecast is differentiable end-to-end, one can *fit* the initial state (or parameters) so a forecast matches a target — the essence of 4D-Var and of parameter estimation. `ex05_optimize_ic.py` adjusts a few initial-temperature amplitudes by gradient descent so the forecast kinetic energy hits a prescribed target (Figure 3):

```
loss_and_grad = jax.jit(jax.value_and_grad(lambda th: (ke_of(th) - target)**2))
for it in range(steps):
    loss, g = loss_and_grad(theta)
    theta = theta - lr * g / (jnp.linalg.norm(g) + 1e-12)
```

Swap the metric for an observation-misfit plus a background term and this is variational data assimilation; swap θ for physics parameters and it is gradient-based calibration.

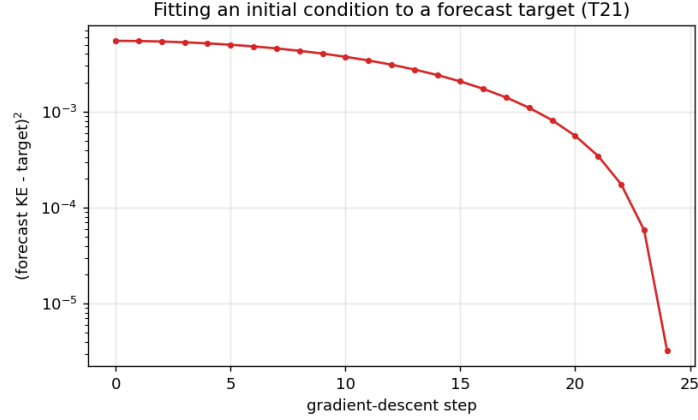


Figure 3: Example 5: gradient descent through the model fits an initial condition so that the forecast kinetic energy reaches a target (squared misfit vs. iteration).

12.6 Example 6 — optimal perturbations / singular vectors (jvp+vjp)

The tangent-linear propagator $M = \partial(\text{rollout})/\partial x$ and its adjoint $M^\top$ are exactly `jax.jvp` and `jax.vjp`. The leading *singular vector* — the initial perturbation that grows fastest over a window, central to predictability and ensemble design — is the top eigenvector of $M^\top M$, found by power iteration using only these two:

```
_, vjp_fn = jax.vjp(g, x0) # adjoint M^T
apply_M = lambda dx: jax.jvp(g, (x0,), (dx,))[1] # tangent-linear M
dx = random_unit_vector
for _ in range(iters):
    Mdx = apply_M(dx); growth = norm(Mdx) # ||M dx|| with ||dx|| = 1
    dx = vjp_fn(Mdx)[0]; dx = dx / norm(dx) # dx <- M^T M dx, renormalise
```

`ex06_singular_vector.py` converges to an optimal 12-hour growth factor ~ 6 for a plain- L^2 norm — again with no hand-written adjoint model.

13 Numerical caveats when differentiating

Autodiff is exact for what the code computes, but the *model* has properties that shape how you should use the gradients.

- **Chaos limits the useful lead time of a gradient.** The atmosphere is chaotic: initial-condition perturbations grow exponentially, so $\partial(\text{forecast})/\partial x_0$ grows like $e^{\lambda t}$ and becomes enormous and noisy beyond the predictability horizon. Gradients over hours-to-days are informative; gradients over weeks are dominated by the leading Lyapunov vector and are numerically fragile. For long windows, regularise, shorten the window, or use ensemble/stochastic estimators.
- **Reverse-mode memory grows with rollout length.** An adjoint stores intermediate states to replay them backwards; a long `lax.scan` can exhaust memory. Wrap the scan body in `jax.checkpoint` (gradient checkpointing) to recompute rather than store — trading compute for memory.
- **Differentiate real (grid-space) inputs, not complex spectral coefficients.** The spectral state is complex, and the model is not a holomorphic function of it, so differentiating with respect to raw coefficients invites subtleties. All the examples differentiate with

respect to *real grid-space* fields (an initial temperature field, real perturbation amplitudes), which keeps everything real and well-defined.

- **Forward-mode and a 0/0 in the tracer limiter.** The positive-definite tracer vertical-advection limiter divides by inter-level differences; with an *identically zero* tracer these are 0/0 (guarded to $\sim 10^{-308}$), which forward mode (`jacfwd/jvp`) turns into NaN by dividing tangents by that tiny number. The fix (used in `_model.rest_state`) is to seed the tracer with a tiny, vertically varying, strictly positive field — physically inert in a dry core, but it keeps the limiter well-conditioned. A good reminder that autodiff exposes the smoothness (or not) of every branch the code takes.
- **Float64.** Reverse-mode through stiff spectral dynamics in single precision is even worse than the forward model; keep `JAX_ENABLE_X64`.

14 Extending the core: non-differentiable kernels

If you must call a non-differentiable or external routine (e.g. a lookup table, a legacy kernel) inside the model, wrap it with `jax.custom_vjp` to supply a hand-written derivative, or with `jax.pure_callback` for a black box; the surrounding model stays differentiable. This is the recommended path if a transform library without JAX support is ever substituted for `s2fft`.

API quick reference

symbol	purpose
<code>SpectralState / GridState</code>	prognostic state containers (pytrees)
<code>spectral_to_grid(S, cfg)</code>	inverse SHT $\rightarrow$ (<code>GridState</code> , <code>GridGradients</code>)
<code>grid_to_spectral(G, cfg)</code>	forward SHT $\rightarrow$ <code>SpectralState</code>
<code>prebuild_kernels(L, "gl")</code>	eagerly build transform kernels (before jit)
<code>get_spectral_tendencies(...)</code>	primitive-equation RHS in spectral space
<code>advance(...)</code>	one dynamics-only IMEX-RK step
<code>advance_with_tendencies(..., phys_tends, spec_tends)</code>	step + physics/stochastic increment
<code>init_semi_implicit_matrices(...)</code>	build the gravity-wave solve operators
<code>init_diffusion_operators(...)</code>	build hyper-diffusion / Rayleigh damping
<code>compute_pressure_diagnostics(lnps, cfg)</code>	p_s , p_k , Δp , layer means, α

Further reading

- **Spectral transform method & primitive equations:** [Durrán \(2010\)](#); the GFS/Fortran lineage this core ports.
- **Semi-implicit / IMEX time stepping:** [Durrán \(2010\)](#) (Ch. 8); [Ascher et al. \(1997\)](#) for IMEX Runge–Kutta.
- **Idealised forcing:** [Held & Suarez \(1994\)](#).
- **Automatic differentiation:** [Baydin et al. \(2018\)](#) (a survey); [Griewank & Walther \(2008\)](#) (the definitive text); the JAX docs on `grad/jvp/vjp/jacfwd/jacrev/checkpoint`.
- **Adjoint, 4D-Var, singular vectors in NWP:** [Errico \(1997\)](#) (what an adjoint model is and does); [Kalnay \(2003\)](#) (4D-Var); [Palmer et al. \(1998\)](#) (singular vectors and predictability).

- **Differentiable atmospheric models:** Kochkov et al. (2024) (NeuralGCM), a large-scale demonstration of the same idea.

References

- Ascher, U. M., Ruuth, S. J., & Spiteri, R. J. (1997). Implicit-explicit Runge–Kutta methods for time-dependent partial differential equations. *Applied Numerical Mathematics*, 25, 151–167.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. *JMLR*, 18(153), 1–43.
- Durran, D. R. (2010). *Numerical Methods for Fluid Dynamics*, 2nd ed. Springer.
- Errico, R. M. (1997). What is an adjoint model? *Bulletin of the AMS*, 78(11), 2577–2591.
- Griewank, A., & Walther, A. (2008). *Evaluating Derivatives*, 2nd ed. SIAM.
- Held, I. M., & Suarez, M. J. (1994). A proposal for the intercomparison of the dynamical cores of atmospheric general circulation models. *Bulletin of the AMS*, 75(10), 1825–1830.
- Kalnay, E. (2003). *Atmospheric Modeling, Data Assimilation and Predictability*. Cambridge University Press.
- Kochkov, D., et al. (2024). Neural general circulation models for weather and climate. *Nature*, 632, 1060–1066.
- Palmer, T. N., Gelaro, R., Barkmeijer, J., & Buizza, R. (1998). Singular vectors, metrics, and adaptive observations. *Journal of the Atmospheric Sciences*, 55(4), 633–653.
- Thuburn, J. (1996). Multidimensional flux-limited advection schemes. *Journal of Computational Physics*, 123, 74–83.